Observability-Driven Root Cause Analysis of Industrial Sensor Data Pipelines

Grounded, Explainable, and Human-in-the-Loop Diagnosis using Large Language Models

Summary

When a data pipeline fails, engineers typically search through logs, metrics and workflow graphs by hand to find out why. Monitoring tools can tell them that something broke, but rarely why. Large language models can reason over this evidence, but without grounding they produce plausible yet wrong answers.

My thesis designs, implements and evaluates an end-to-end system that closes this gap. It records pipeline failures as structured incidents, retrieves similar past incidents with vector search (RAG), and has an LLM produce an evidence-based diagnosis with corrective and preventive actions. Low-confidence or unsupported results are escalated to a human reviewer instead of being resolved automatically.

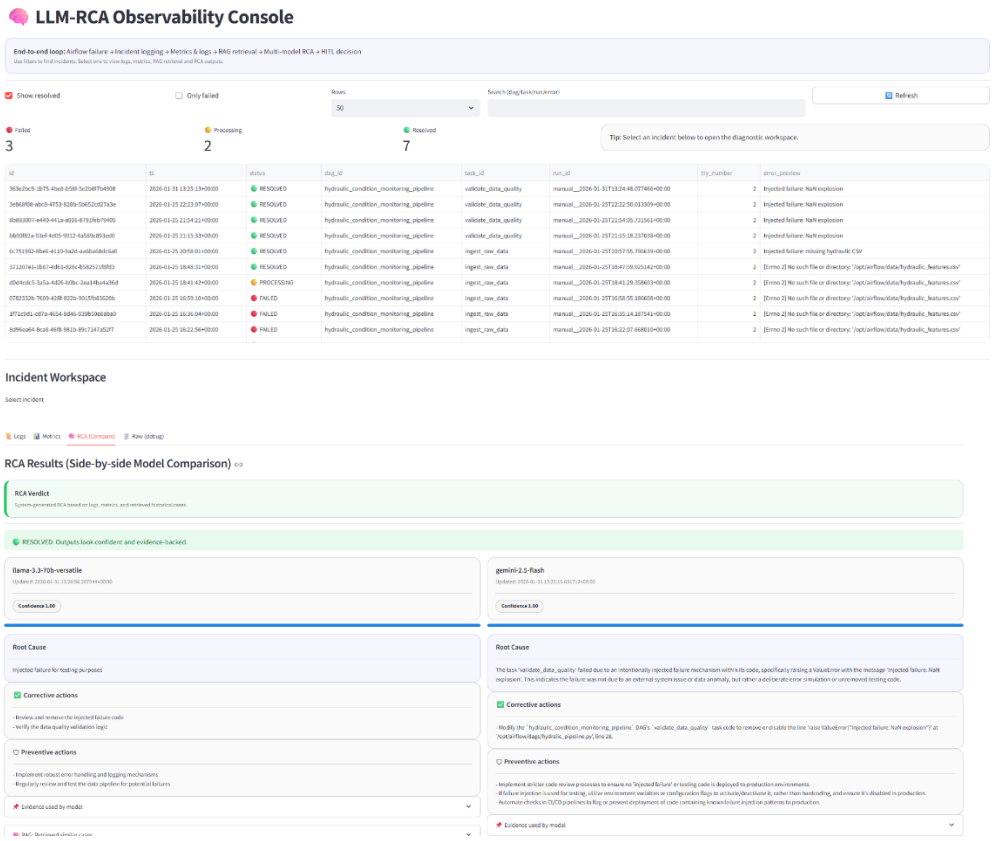


Figure 1. The RCA workspace presenting a diagnosis together with its supporting evidence.

1. Problem and Research Questions

Pipelines fail routinely through schema drift, corrupted data or logic errors. Diagnosis is slow, depends on individual experience, and past incident knowledge is scattered across tickets and people. Standalone LLMs are risky here because they lack knowledge of an organisation's own systems and may hallucinate a confident but wrong root cause.

RQ1 How effectively can an observability-driven system detect pipeline failures and assemble enough structured context for reliable RCA?

RQ2 To what extent does Retrieval-Augmented Generation improve accuracy, evidence grounding and actionability of LLM-based RCA?

RQ3 How do different LLMs and human-in-the-loop mechanisms affect reliability, trust and operational suitability?

2. System Design

The central design principle is the separation of pipeline execution from diagnostic reasoning, with the LLM embedded in a deterministic, stateful workflow rather than left to act freely.

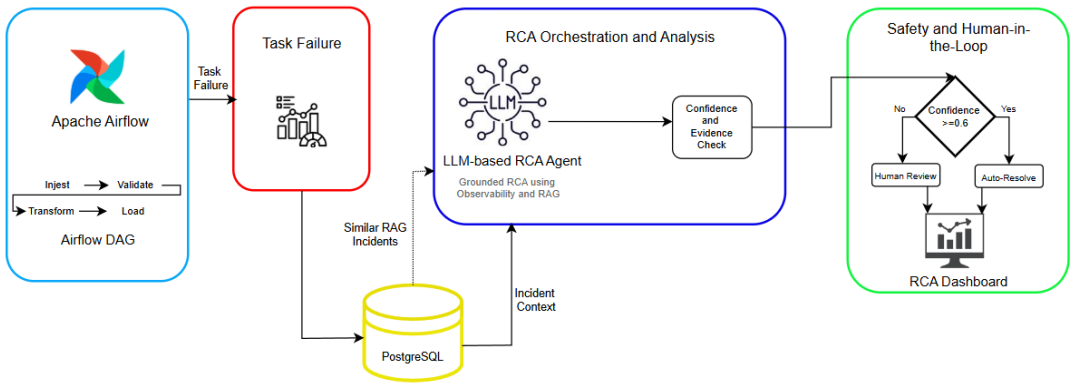


Figure 2. System architecture from pipeline execution to human review.

Pipeline	Apache Airflow DAG with seven stages: ingestion, schema validation, data-quality validation, preprocessing, feature engineering, aggregation, loading.
Observability	Failure callbacks capture logs, stack traces, task metadata and structured metrics. An incident is created only after all retries are exhausted, exactly once per failure.

Storage	PostgreSQL holds incidents, logs, metrics and RCA results. pgvector stores embeddings of historical cases.
RCA agent	A LangGraph workflow assembles incident context, retrieves similar past cases and prompts the LLM.
Reasoning	Gemini 2.5 Flash and Llama 3.3 70B receive identical context and return a structured result: root cause, evidence, corrective and preventive actions, confidence.
Human review	Streamlit dashboard for incidents, logs, metrics and model comparison. Results below the confidence or evidence threshold go to a human.

3. Experimental Setup

The system was evaluated on the UCI Condition Monitoring of Hydraulic Systems dataset, a real industrial sensor dataset, running in a Docker Compose environment. Six realistic failure types were injected into specific pipeline stages so that every diagnosis could be checked against a known cause.

Failure type	Failed Airflow task	Injected runs	Incident detected
Missing input file	`ingest_raw_data`	1	1
Schema Drift (PS3 missing)	`validate_schema`	1	1
NaN explosion	`Validate_data_quality`	2	2
Preprocessing collapse	`preprocess_signals`	1	1
Divide-by-zero	`feature_engineering`	2	2
DB constraint violation	`load_to_postgres`	1	1
Total	-	8	8

Table 1. Injected failures and incident detection.

4. Key Results

RQ1: Detection and observability

- All 8 injected failures were detected with exactly one incident record each, with no misses and no duplicates.
- Execution logs and stack traces were captured for all 19 incidents. Structured metrics were available for 10 of 19, because early-stage failures such as a missing input file occur before metrics exist.

RQ2: RAG-grounded diagnosis

- The retrieval step returned the matching historical failure type in the examined cases, for example a schema-drift case for a schema-drift incident.
- Every completed RCA matched the injected failure and its task, cited evidence from logs and metrics, and contradicted no observed data. No hallucinated diagnoses were observed.

RQ3: Model choice and human oversight

Model	Runs	Avg. latency	Avg. confidence
Gemini 2.5 Flash (commercial)	14	6.7 s	1.00
Llama 3.3 70B (open-weight)	10	2.1 s	0.99

- Given identical context, both models produced qualitatively equivalent diagnoses. The open-weight model was about 3.2 times faster, so context quality mattered more than model choice.
- 13 of 16 incidents (81.3%) were auto-resolved. The 3 escalated cases were caused by model-endpoint failures, and in each the system failed closed: no partial or speculative diagnosis was saved.

5. Contributions

- End-to-end architecture for automated RCA covering detection, context assembly, LLM reasoning and validation.
- Observability-driven context model that treats structured metrics as low-entropy signals constraining generative reasoning.
- RAG-based incident memory that reuses historical failure knowledge through vector similarity search.
- Controlled comparison of commercial and open-weight LLMs under identical orchestration.
- Human-in-the-loop escalation design that fails safe when evidence or confidence is insufficient.

6. Limitations and Outlook

The evaluation used a small, controlled set of injected failures on a single node, so the results show feasibility rather than production-scale reliability. The individual effect of RAG was not isolated in an ablation study, and confidence scores were self-reported by the models. Natural next steps are broader failure coverage, a larger labelled incident set, systematic diagnosis-quality scoring, an ablation with and without RAG, and evaluation under production-like load.

7. Skills Demonstrated

Data engineering	Airflow DAG design, failure handling and retries, data-quality validation, PostgreSQL schema design
Applied AI	RAG with pgvector, LangGraph agent workflows, prompt design, multi-LLM evaluation (Gemini, Llama via Groq)
Reliability thinking	Observability, idempotent incident creation, confidence thresholds, fail-closed behaviour
Engineering practice	Docker Compose environment, Streamlit UI, reproducible experiments with ground truth